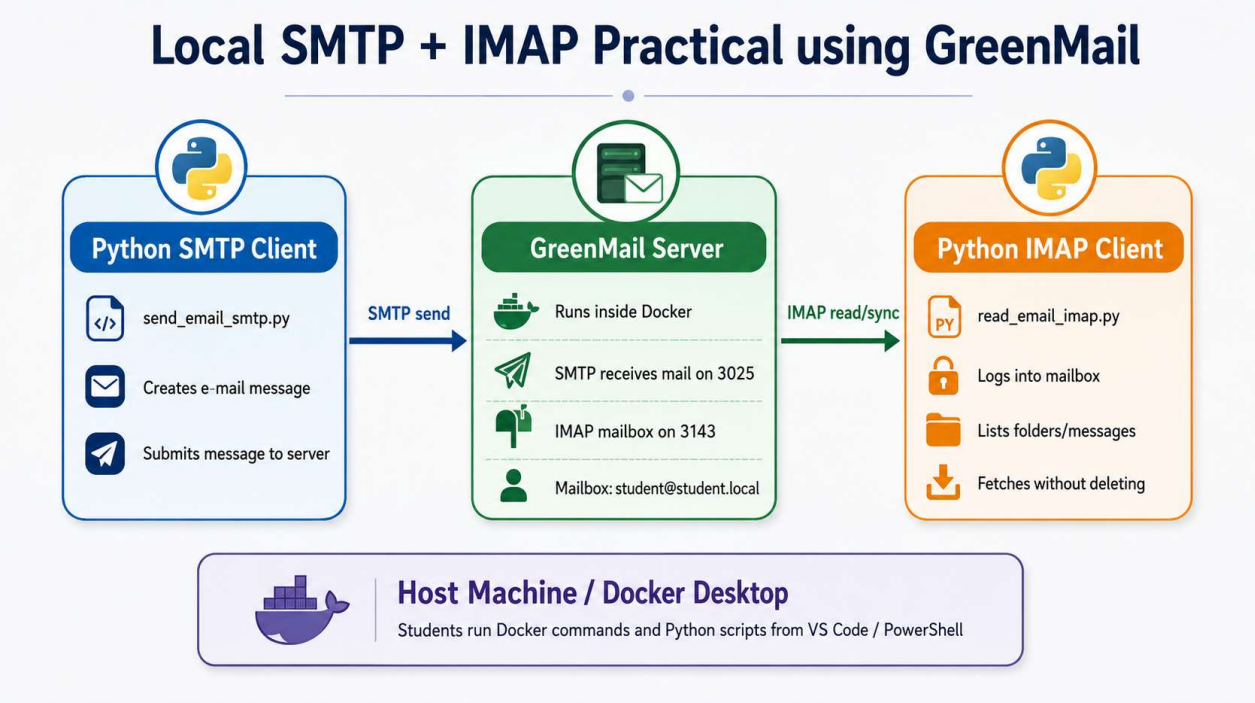
Practical: Local E-mail Server with SMTP and IMAP using GreenMail

Foundation for Python Network Programming | Chapter 15: IMAP

Student Practical Handout



Purpose of this practical

In this practical, students run a local test mail server using Docker. The server supports SMTP, which sends e-mail into the server, and IMAP, which allows a client to view, search, organise, and retrieve messages from the mailbox while the messages remain on the server.

This practical is similar to the POP3 practical, but the retrieval side now uses IMAP. The key idea is that IMAP is not only a download protocol. It supports folders, flags, searching, and selective fetching of headers or message bodies.

> **Teaching note:** Use this practical after the POP3 practical so students can directly compare POP3 download behaviour with IMAP server-side mailbox management.

Protocol context

Protocol	Role in this practical	Local port	Python module
SMTP	Sends the e-mail into GreenMail	3025	smtplib
IMAP	Reads, searches, and manages mailbox messages	3143	imaplib
POP3	Available for comparison only	3110	poplib
GreenMail API/Web	Optional service for checking the server	8080	Browser/API

Learning outcomes

1. Start a local mail server using Docker.

2. Send an e-mail message to a local mailbox using SMTP.

3. Connect to the same mailbox using IMAP.

4. List IMAP folders and select the INBOX folder.

5. Search for messages stored in the mailbox.

6. Retrieve selected headers and the message body using Python.

7. Explain how IMAP differs from POP3.

Required software

Docker Desktop installed and running.

Visual Studio Code or another code editor.

Python installed and available from the terminal.

PowerShell terminal on Windows.

Part 1: Start the GreenMail server

Open PowerShell in VS Code or a normal Windows PowerShell terminal. Run the following command:

```
docker run --rm `
 --name greenmail-imap-lab `
 -p 3025:3025 `
 -p 3110:3110 `
 -p 3143:3143 `
 -p 8080:8080 `
 -e GREENMAIL_OPTS="-Dgreenmail.setup.test.all -Dgreenmail.hostname=0.0.0.0 -
Dgreenmail.users=student:password@student.local -Dgreenmail.verbose" `
 greenmail/standalone:2.1.8
```

Command part	Function
docker run	Starts a new Docker container.
--rm	Removes the container automatically when it stops. This keeps the lab environment clean.
--name greenmail-imap-lab	Gives the container a clear name so that students can recognise it in docker ps.
-p 3025:3025	Maps GreenMail SMTP to the local computer. Students send mail to this port.
-p 3143:3143	Maps GreenMail IMAP to the local computer. Students read and manage mail through this port.
-p 3110:3110	Maps POP3 for comparison with the previous practical.
-p 8080:8080	Exposes the GreenMail web/API interface.
-Dgreenmail.setup.test.all	Starts common GreenMail test services, including SMTP, POP3, and IMAP.
-Dgreenmail.hostname=0.0.0.0	Allows GreenMail to listen on all network interfaces inside the Docker container.
-Dgreenmail.users=student:password@student.local	Creates a test mailbox. Username: student. Password: password. E-mail address: student@student.local.

Part 2: Confirm that the container is running

Open a second PowerShell terminal and run:

```
docker ps
```

You should see a container called greenmail-imap-lab. You should also see mapped ports including 3025, 3110, 3143, and 8080. If the container is not running, return to Part 1 and check the Docker command carefully.

Part 3: Create a Python virtual environment

In VS Code, create a new folder for the practical, for example:

```
imap_greenmail_lab
```

Open a terminal in that folder and create a virtual environment:

```
python -m venv .venv
```

Activate it in PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

Your terminal should now show something similar to:

```
(.venv) PS C:\...\imap_greenmail_lab>
```

Teaching note: The smtplib, imaplib, and email modules used in this practical are included in the Python Standard Library. No external package is required. When a future lab does require a package, use python -m pip install <package-name> so pip installs into the active Python environment.

Part 4: Send an e-mail using SMTP

Create a file called send_email_smtp.py and add the following code:

```
# Import smtplib, which allows Python to send e-mail using SMTP
import smtplib

# Import EmailMessage, which helps us build a properly formatted e-mail message
from email.message import EmailMessage

# Create a new e-mail message object
message = EmailMessage()

# Set the sender address. This can be a test address because we use a local server.
message["From"] = "sender@example.com"

# Set the recipient address. This must match the GreenMail mailbox.
message["To"] = "student@student.local"

# Set the subject line of the message
message["Subject"] = "IMAP Practical Test Message"

# Set the body text of the message
message.set_content(
    "Hello student,\n\n"
    "This message was sent using SMTP and will be read using IMAP.\n"
    "IMAP keeps the message on the server and lets us search, flag, and fetch it.\n\n"
    "Regards,\n"
    "Python SMTP Client"
)

# Define the SMTP server details
smtp_host = "localhost"
smtp_port = 3025

# Connect to the local GreenMail SMTP server
with smtplib.SMTP(smtp_host, smtp_port) as smtp:
    # Print the SMTP conversation for learning purposes
    smtp.set_debuglevel(1)

    # Send the message into the GreenMail server
    smtp.send_message(message)

print("E-mail sent successfully.")
```

Run the script:

```
python send_email_smtp.py
```

Expected result:

```
E-mail sent successfully.
```

Code line or block	Explanation
import smtplib	Imports Python's SMTP library. SMTP is used to send e-mail from a client to a mail server.
from email.message import EmailMessage	Imports a helper class for creating properly formatted e-mail messages.
message = EmailMessage()	Creates a new e-mail message object.
message["To"] = "student@student.local"	Sets the recipient mailbox created in GreenMail.
message.set_content(...)	Sets the plain-text body of the e-mail.
smtp_host = "localhost"	The mail server is running locally through Docker port mapping.
smtp_port = 3025	The SMTP service is mapped to local port 3025.
with smtplib.SMTP(...) as smtp:	Opens an SMTP connection and automatically closes it when done.
smtp.set_debuglevel(1)	Prints the SMTP conversation to the terminal for learning purposes.
smtp.send_message(message)	Sends the e-mail message into the GreenMail server.

Part 5: Read the e-mail using IMAP

Create a second file called read_email_imap.py and add the following code:

```
# Import imaplib, which allows Python to connect to an IMAP server
import imaplib

# Import the email module so that we can parse the raw e-mail message
import email

# Define the IMAP server details
imap_host = "localhost"
imap_port = 3143

# Define the mailbox login details
username = "student"
password = "password"

# Connect to the local GreenMail IMAP server
mailbox = imaplib.IMAP4(imap_host, imap_port)

try:
    # Log in to the mailbox
    mailbox.login(username, password)

    # List folders available on the IMAP server
    status, folders = mailbox.list()
    print("--- Folders on the server ---")
    for folder in folders:
        print(folder.decode())

    # Select the INBOX folder in read-only mode.
    # IMAP is stateful: later commands operate on the selected folder.
    status, select_data = mailbox.select("INBOX", readonly=True)
    message_count = int(select_data[0])
    print(f"\nINBOX contains {message_count} message(s).")

    # Search for all messages in the selected folder
    status, search_data = mailbox.search(None, "ALL")
    message_ids = search_data[0].split()

    if not message_ids:
```

```
        print("No messages found. Run send_email_smtp.py first.")
    else:
        # Retrieve the most recent message number
        latest_id = message_ids[-1]
        print(f"Fetching message number: {latest_id.decode()}")

        # BODY.PEEK[] retrieves the message without marking it as read
        status, fetch_data = mailbox.fetch(latest_id, "(BODY.PEEK[])")

        # The returned data includes protocol metadata and the raw message bytes
        raw_message = None
        for item in fetch_data:
            if isinstance(item, tuple):
                raw_message = item[1]

        # Convert the raw bytes into an e-mail message object
        message = email.message_from_bytes(raw_message)

        # Display selected headers
        print("\n--- Message Headers ---")
        print("From:", message["From"])
        print("To:", message["To"])
        print("Subject:", message["Subject"])

        # Display the message body
        print("\n--- Message Body ---")
        if message.is_multipart():
            for part in message.walk():
                if part.get_content_type() == "text/plain":
                    print(part.get_payload(decode=True).decode())
        else:
            print(message.get_payload(decode=True).decode())

finally:
    # Always close the IMAP session properly
    mailbox.logout()
```

Run the script:

```
python read_email_imap.py
```

Expected result:

```
--- Folders on the server ---
...
INBOX contains 1 message(s).
Fetching message number: 1

--- Message Headers ---
From: sender@example.com
To: student@student.local
Subject: IMAP Practical Test Message

--- Message Body ---
```

```
Hello student,
This message was sent using SMTP and will be read using IMAP.
IMAP keeps the message on the server and lets us search, flag, and fetch it.
```

Code line or block	Explanation
import imaplib	Imports Python's IMAP library. IMAP is used to access and manage mailbox messages on a server.
mailbox = imaplib.IMAP4(...)	Opens an IMAP connection to the local GreenMail server.
mailbox.login(username, password)	Authenticates to the mailbox using the GreenMail test user.
mailbox.list()	Asks the server for available folders, such as INBOX.
mailbox.select("INBOX", readonly=True)	Selects the INBOX folder. The readonly option prevents accidental message changes.
mailbox.search(None, "ALL")	Searches the selected folder and returns message numbers for matching messages.
message_ids[-1]	Selects the most recent message number from the search result.
mailbox.fetch(..., "(BODY.PEEK[])")	Retrieves the full message without setting the read/Seen flag.
email.message_from_bytes(...)	Parses the raw e-mail bytes into a Python e-mail object.
mailbox.logout()	Closes the IMAP session properly.

Part 6: Fetch headers only

IMAP can retrieve only the parts of a message that are needed. This is useful when a mailbox contains large messages or attachments. Create a third file called imap_headers_only.py:

```
import imaplib

mailbox = imaplib.IMAP4("localhost", 3143)
try:
    mailbox.login("student", "password")
    mailbox.select("INBOX", readonly=True)

    # Search for all messages in the selected folder
    status, search_data = mailbox.search(None, "ALL")
    message_ids = search_data[0].split()

    # Fetch only the From and Subject header fields for each message
    for msg_id in message_ids:
        status, data = mailbox.fetch(
            msg_id,
            "(BODY.PEEK[HEADER.FIELDS (FROM SUBJECT)])"
        )
        print(f"\nMessage {msg_id.decode()}")
        for item in data:
            if isinstance(item, tuple):
                print(item[1].decode().strip())
finally:
    mailbox.logout()
```

Run the script:

```
python imap_headers_only.py
```

Teaching note: This demonstrates why IMAP is useful for modern mail clients. A client can show a mailbox summary without downloading every full message and every attachment.

Part 7: Search messages on the server

IMAP supports server-side searching. This means the client can ask the server to find matching messages without downloading the whole mailbox first. Modify the search line in read_email_imap.py and test the examples below:

Search example	Meaning
mailbox.search(None, "ALL")	Return all messages in the selected folder.
mailbox.search(None, "SUBJECT", "IMAP")	Return messages whose Subject header contains IMAP.
mailbox.search(None, "FROM", "sender@example.com")	Return messages from the given sender.
mailbox.search(None, "UNSEEN")	Return messages not marked as seen/read.

Part 8: What you should observe

You should observe that the same message is first sent into the local mail server using SMTP and then accessed from the mailbox using IMAP:

Python SMTP client -> GreenMail mailbox -> Python IMAP client

The key conceptual difference is that SMTP is used for sending and relaying e-mail, while IMAP is used to access and manage messages that remain on the server. Compared with POP3, IMAP is better suited to multiple devices because folders, flags, and message state can remain synchronised on the server.

Part 9: POP3 vs IMAP comparison

Feature	POP3	IMAP
Main purpose	Download messages from a mailbox.	Access and manage messages stored on the server.
Folders	Usually focuses on one mailbox.	Supports folders such as INBOX, Sent, Drafts, and custom folders.
Message state	Often local to one device after download.	Server can store flags such as read, answered, deleted, and flagged.
Searching	Client may need to download messages first.	Server-side searching is supported.
Multiple devices	Can become awkward because messages may be scattered or duplicated.	Designed for synchronised access from multiple devices.
Selective fetching	Limited.	Can fetch only headers, selected parts, or full messages.

Part 10: Student reflection questions

1. What protocol was used to send the message into the mail server?
2. What protocol was used to access the message from the mailbox?
3. Why is SMTP not used to read mail from the mailbox?
4. What does mailbox.list() return?
5. Why must the client select a folder before searching or fetching messages?
6. What does mailbox.search(None, "ALL") return?
7. Why does BODY.PEEK[] matter in this practical?

8. What is the difference between fetching only headers and fetching the full message?
9. Why is IMAP useful when a person reads the same mailbox from a phone, laptop, and webmail?
10. Give two differences between POP3 and IMAP.

Part 11: Troubleshooting

Problem	Action
Docker command not found	Check that Docker Desktop is installed and running. Run docker --version.
Port already in use	Stop the existing container using docker stop greenmail-imap-lab, then start the container again. If you used the POP3 practical container name, also run docker ps and stop greenmail-pop-lab if it is still using the same ports.
IMAP connection refused	Check that the container is running using docker ps and confirm that port 3143 is mapped.
Mailbox has zero messages	Run python send_email_smtp.py first. Then run python read_email_imap.py again.
Authentication failed	Check that the Docker command created the user as student:password@student.local. The username is student and the password is password.
Wrong Python environment	Check the active Python interpreter using python -c "import sys; print(sys.executable)". Use python -m pip install <package-name> for packages in future labs.

Summary

This practical demonstrates a complete local e-mail flow. SMTP is used to send e-mail into the GreenMail server. IMAP is used to access the mailbox, list folders, search messages, and fetch either headers or full message content. GreenMail allows the practical to be completed locally without sending real e-mail over the Internet.